Table 13.1 Django Debug Toolbar panels

Panel	Description
History	History of requests made, which also includes the ability to view the debug info for past visits
Versions	Shows the versions of Django, Python, and installed apps (if possible)
Timer	Shows page load time information, including CPU time, system time, and a graph showing the timing for each step in processing the page
Settings	Shows your current project configuration values
Headers	Request and response headers along with WSGI environment information
Request	Information about the HTTP request, including the view function invoked, any cookies, session data, and data for the GET or POST
SQL	SQL queries run to get the page's data along with timing information
Static files	Information on the static files used on the page along with all the possible files available to the {% static %} tag
Templates	Shows the template path and all templates involved in rendering this page
Cache	Whether the cache got used along with hit-and-miss counts and response times
Signals	All the Django signals registered and their associated handlers
Intercept redirects	If a page issues a redirect, the toolbar catches it and shows a page with the resulting link (panel off, by default)
Profiling	Complete profile information on the request, including all the functions called and their timings (panel off by default)

If you click on a panel to open it, you can close it again with the cancellation *X* in the top right corner. The panel I personally find most useful is the one showing the SQL queries. Examining the details for the `musicians()` view reveals that two queries got run. The first counted all the musicians in the database, while the second queried using the musicians limiting the number of results. This two-step process occurs because the musicians view uses the `Paginator` object; the first call checks how many items there are in the entire database, while the second limits how many come back in a page.

The most likely performance bottleneck you'll have on a page is from the database. One of the downsides of an ORM is you're not always cognizant of your query's complexity. If you're finding your page load is slow, the Time and SQL panels can help you get to the bottom of it.

13.2 Tool libraries

Django is a large framework with lots of features, but there are always situations where there could be more. Often, these features are small and not worthy of their own library. As a result, there are several third-party libraries that are collections of miscellaneous tools. This section covers two of these tool kits: Django Extensions and Django Awl.